Inheritance

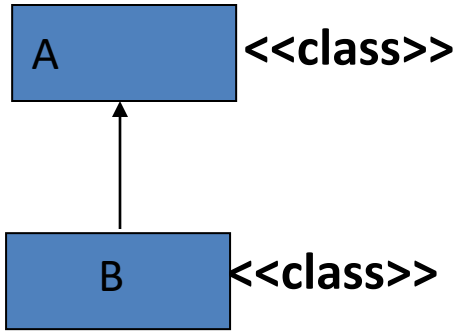
INHERITANCE BASICS

1. **Reusability is achieved by *INHERITANCE***
2. **Java classes Can be Reused by extending a class. Extending an existing class is for reusing properties of the existing classes.**
3. **The class whose properties are extended is known as *super or base or parent class*.**
4. **The class which extends the properties of super class is known as *sub or derived or child class***
5. ***A class can either extends another class or can implement an interface***

class B extends A { ….. }

A super class

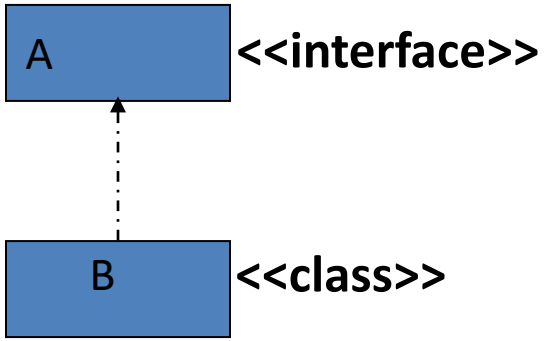
B sub class



class B implements A { ….. }

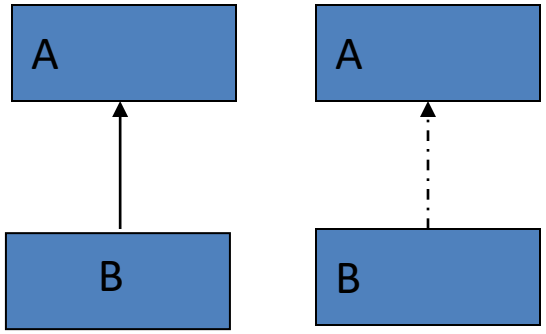
A interface

B sub class

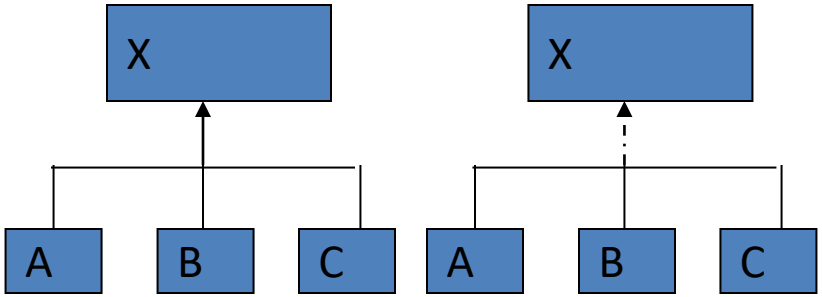


Various Forms of Inheritance

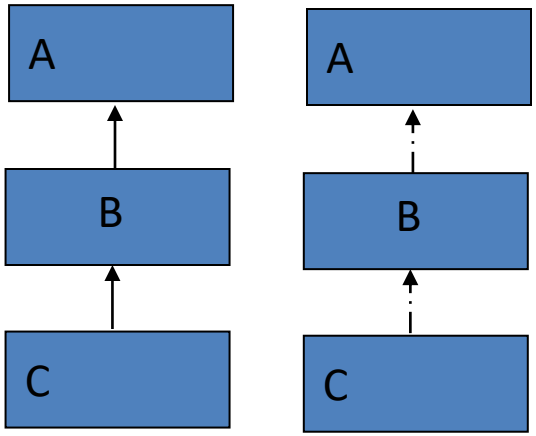
Single Inheritance



Hierarchical Inheritance



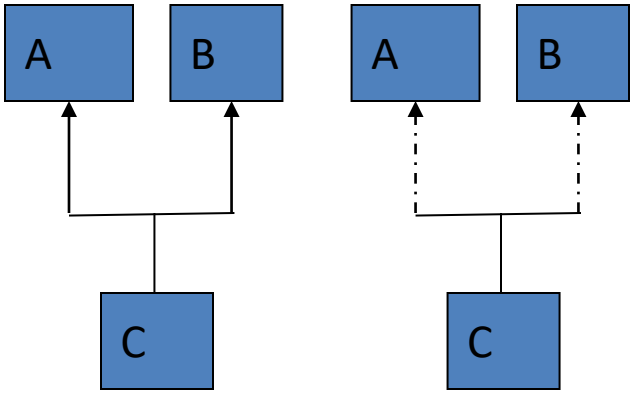
MultiLevel Inheritance



NOT SUPPORTED BY JAVA

Multiple Inheritance

SUPPORTED BY JAVA



Forms of Inheritance

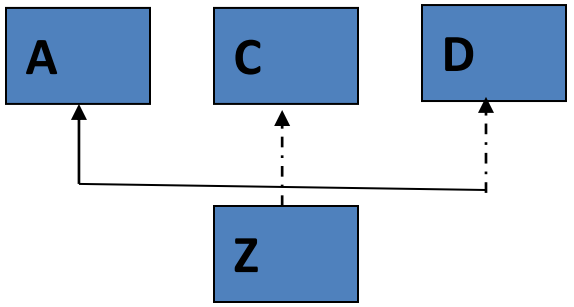
- **Mulitiple Inheritance can be implemented by implementing multiple interfaces not by extending multiple classes**

Example :

class Z extends A implements C , D

{ ……}

OK



class Z extends A ,B

{

WRONG

}

OR

class Z extends A extends B

{

WRONG

}

Defining a Subclass

Syntax :

```
class <subclass name> extends <superclass name>
{
 variable declarations;
 method declarations;
}
```

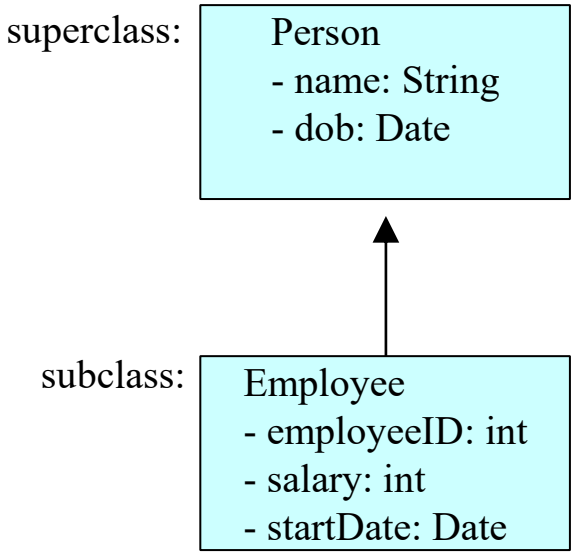
- **extends keyword signifies that properties of the super class are extended to sub class**
- **Sub class will not inherit private members of super class**

Access Control

Access Modifiers → / Access Location ↓	public	protected	private
Same Class	Yes	Yes	Yes
sub classes in same package	Yes	Yes	No
Other Classes in Same package	Yes	Yes	No
Subclasses in other packages	Yes	Yes	No
Non-subclasses in other packages	Yes	No	No

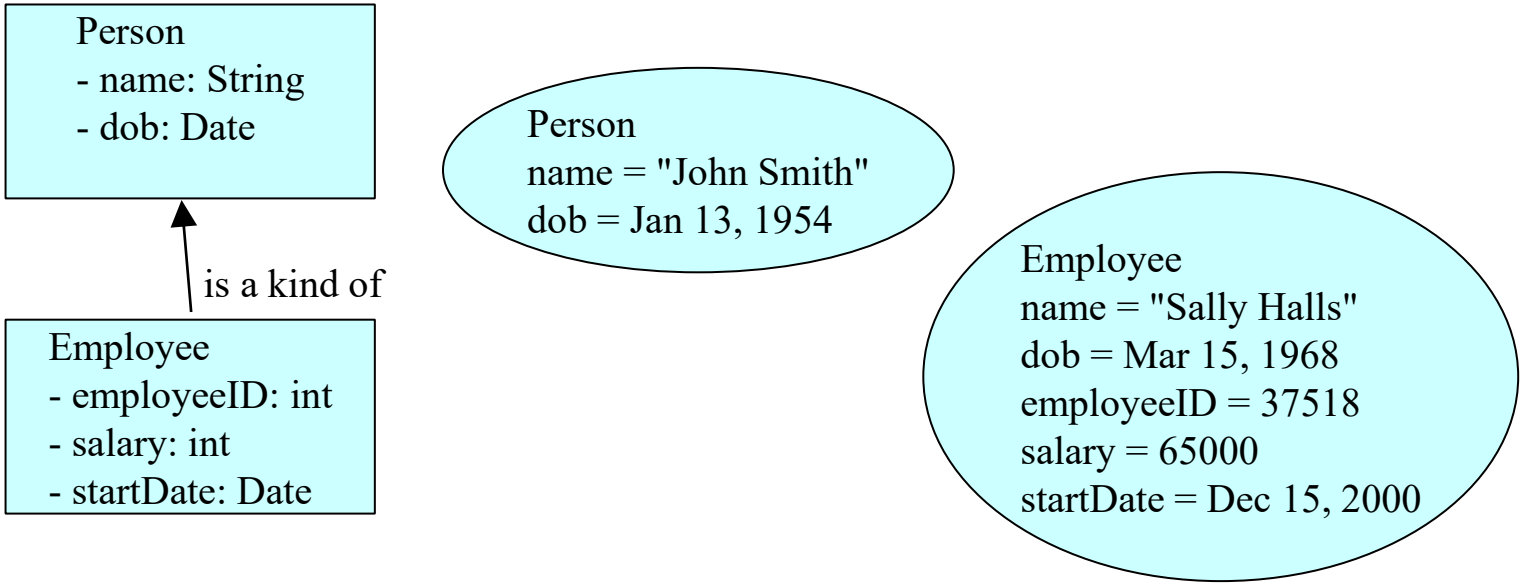
Terminology

- Inheritance is a fundamental Object Oriented concept
- A class can be defined as a "subclass" of another class.
  - The subclass inherits all data attributes of its superclass
  - The subclass inherits all methods of its superclass
  - The subclass inherits all associations of its superclass
- The subclass can:
  - Add new functionality
  - Use inherited functionality
  - Override inherited functionality



What really happens?

- When an object is created using new, the system must allocate enough memory to hold all its instance variables.
  - This includes any inherited instance variables
- In this example, we can say that an Employee "is a kind of" Person.
  - An Employee object inherits all of the attributes, methods and associations of Person



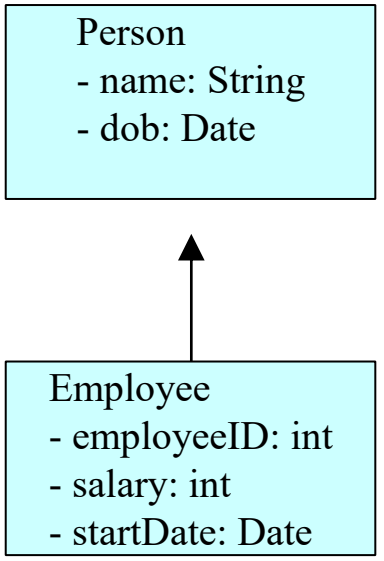
Inheritance in Java

- Private members are not inherited in subclass

```
public class Person
{
  private String name;
  private Date dob;
  [...]
```

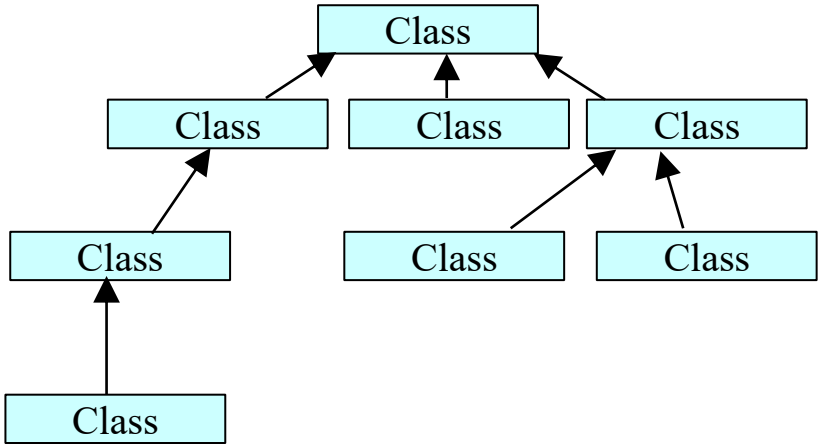
```
public class Employee extends Person
{
  private int employeID;
  private int salary;
  private Date startDate;
  [...]
```

```
Employee anEmployee = new Employee();
```



Inheritance Hierarchy

- Each Java class has one (and only one) superclass.
  - C++ allows for multiple inheritance
- Inheritance creates a class hierarchy
  - Classes higher in the hierarchy are more general and more abstract
  - Classes lower in the hierarchy are more specific and concrete
- There is no limit to the number of subclasses a class can have
- There is no limit to the depth of the class tree.



Inheritance Basics

1. Whenever a subclass object is created, super class constructor is called first.
2. If super class constructor does not have any constructor of its own OR has an unparametrized constructor then it is automatically called by Java Run Time by using call super()
3. If a super class has a parameterized constructor then it is the responsibility of the sub class constructor to call the super class constructor by call

   super(<parameters required by super class>)

4. Call to super class constructor must be the first statement in sub class constructor

Inheritance Basics

When super class has a Unparametrized constructor

```
class A
{
A()
{
System.out.println("This is constructor of class A");
}
} // End of class A
class B extends A
{
B()
{
super();
System.out.println("This is constructor of class B");
}
} // End of class B
```

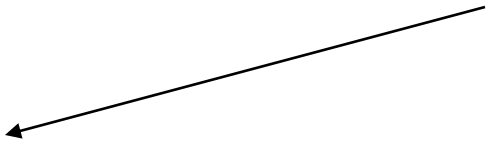
```
class inhtest
{  public static void main(String args[])
   {   B b1 = new B();
} }
```

OUTPUT

This is constructor of class A

This is constructor of class B

Optional



Without super() call

```
class A
{
A()
{
System.out.println("This is class A");
} }

class B extends A
        {
        B()
        {
        System.out.println("This is class B");
        } }

class inherit1
{
public static void main(String args[])
{
B b1 = new B();
} }
```

output

This is class A

This is class B

private constructor

```
class A
{
private A()
{
System.out.println("This is class A");
}
}
class B extends A
{
B()
{
System.out.println("This is class B");
}
}
class inherit2
{
public static void main(String args[])
{
B b1 = new B();
}
}
```

```
/*
C:\Java>javac inherit2.java
inherit2.java:12: A() has private access in A
{
^
1 error
*/
```

```
class A
{
private A()
{
System.out.println("This is class A");
}
A()
{
System.out.println("This is class A");
}
}
class B extends A
{
B()
{
System.out.println("This is class B");
}
}
class inherit2
{
public static void main(String args[])
{
B b1 = new B();
} }
```

```
/*
E:\Java>javac inherit2.java
inherit2.java:7: A() is already defined in A
A()
^
inherit2.java:16: A() has private access in A
{
^
2 errors
*/
```

When Super class has a parametrized constructor.

```
class A
{
private int a;
A( int a)
{
this.a =a;
System.out.println("This is constructor of class A");
}
 }
class B extends A
{
private int b;
private double c;
B(int b,double c)
{
this.b=b;
this.c=c;
System.out.println("This is constructor of class B");
}
}
```

B b1 = new B(10,8.6);

```
D:\java\bin>javac inhtest.java
inhtest.java:15: cannot find
symbol
symbol  : constructor A()
location: class A
{
^
1 errors
```

```
class A
{
private int a;
A( int a)
{
this.a =a;
System.out.println("This is constructor of
class A");
} }
class B extends A
{
private int b;
private double c;
B(int a,int b,double c)
{
super(a);
this.b=b;
this.c=c;
System.out.println("This is constructor of
class B");
} }
```

B b1 = new B(8,10,8.6);

OUTPUT

This is constructor of class A

This is constructor of class B

```
class A
{
private int a;
protected String name;
A(int a, String n)
{
this.a = a;
this.name = n;
}
void print()
{
System.out.println("a="+a);
}
}
```

```
class B extends A
{
int b;
double c;
B(int a,String n,int b,double c)
{
super(a,n);
this.b=b;
this.c =c;
}
void show()
{
//System.out.println("a="+a);
print();
System.out.println("name="+name);
System.out.println("b="+b);
System.out.println("c="+c);
}
}
```

OutPut

```
E:\Java>java xyz3
a=10
name=OOP
b=8
c=10.56
```

```
class xyz3
{   public static void main(String args[])
    {   B b1 = new B(10,"OOP",8,10.56);
        b1.show();
}   }
```

'a' is private in class A

Call to print() from super class A

Accessing name field from super class (super.name)

USE OF super KEYWORD

- Can be used to call super class constrctor

  super();

  super(<parameter-list>);

- Can refer to super class instance variables/Methods

  super.<super class instance variable/Method>

```
class A
{
private int a;
A( int a)
{
this.a =a;
System.out.println("This is constructor of class
A");
}
void print()
{  System.out.println("a="+a);
}
void display()
{
System.out.println("hello This is Display in A");
}   } // End of class A
```

```
class B extends A
{
private int b;
private double c;
B(int a,int b,double c)
{
super(a);
this.b=b;
this.c=c;
System.out.println("This is constructor of
class B");  }
void show()
{
print();
System.out.println("b="+b);
System.out.println("c="+c);
}   } // End of class B
```

```
class inhtest1
{
public static void main(String args[])
{   B b1 = new B(10,8,4.5);
    b1.show();
} }
```

```
/* OutPut
D:\java\bin>java inhtest1
This is constructor of class A
This is constructor of class B
a=10
b=8
c=4.5  */
```

```
class A
{
private int a;
A( int a)
{
this.a =a;
System.out.println("This is constructor of
class A");
}
void show()
{  System.out.println("a="+a);   }
void display()
{
System.out.println("hello This is Display in
A");
} }
```

```
class inhtest1
{  public static void main(String args[])
  {  B b1 = new B(10,8,4.5);
     b1.show();
} }
```

```
class B extends A
{
private int b;
private double c;
B(int a,int b,double c)
{  super(a);
   this.b=b;
   this.c=c;
   System.out.println("This is constructor of
class B");  }
void show()
{   super.show();
   System.out.println("b="+b);
   System.out.println("c="+c);
   display();     // no need super keyword
} }
```

```
/* OutPut
D:\java\bin>java inhtest1
This is constructor of class A
This is constructor of class B
a=10
b=8
c=4.5
hello This is Display in A    */
```

```
class A
{
int a;
A( int a)
{ this.a =a; }
void show()
{
System.out.println("a="+a);
}
void display()
{
System.out.println("hello This is Display in A");
}
}
```

```
class B extends A
{
int b;
double c;
B(int a,int b,double c)
{
super(a);
this.b=b;
this.c=c;
}
void show()
{
//super.show();
System.out.println("a="+a);
System.out.println("b="+b);
System.out.println("c="+c);
}
}
```

```
class inhtest2
{
public static void main(String args[])
{
B b1 = new B(10,20,8.4);
b1.show();
}
}
```

```
/*
D:\java\bin>java inhtest2
a=10
b=20
c=8.4
*/
```

```
class A
{
int a;
A( int a)
{ this.a =a; }
}
```

```
class B extends A
{
// super class variable ‘a’ hides here
int a;
int b;
double c;
B(int a,int b,double c)
{
super(100);
this.a = a;
this.b=b;
this.c=c;
}
void show()
{
System.out.println("Super class a="+super.a);
System.out.println("a="+a);
System.out.println("b="+b);
System.out.println("c="+c);
}
}
```

```
class inhtest2
{
public static void main(String args[])
{
B b1 = new B(10,20,8.4);
b1.show();
}
}
```

```
/* Out Put
D:\java\bin>java inhtest2
Super class a=100
a=10
b=20
c=8.4
*/
```

Date and Time

- Two constructors

  Date() - initializes the object with the current date and time

  Date(long millisec) - accepts one argument that equals the number of milliseconds that have elapsed since midnight, January 1, 1970

An example

```
import java.util.Date;
public class DateDemo
  { public static void main(String args[])
    {       // Instantiate a Date object
      Date date = new Date();
            // display time and date using toString()
      System.out.println(date.toString());
      }
  }
```

Wed Oct 07 09:51:52 IST 2015

Date Formatting using SimpleDateFormat:

```
import java.util.*;
import java.text.*;
public class DateDemo
{
  public static void main(String args[])
  {
     Date dNow = new Date( );
     SimpleDateFormat ft = new SimpleDateFormat ("E yyyy.MM.dd
  'at' hh:mm:ss a zzz");
     System.out.println("Current Date: " + ft.format(dNow));
   }
 }
```

```
Current Date: Sun 2004.07.18 at 04:14:09 PM PDT
```